

	bad	excellent	great	is	magazine	name	of	osfy	quality	the	this
0	0	0	0.657	0.435	0.435	0	0	0	0	0	0.435
1	0.742	0	0	0.387	0.387	0	0	0	0	0	0.387
2	0	0	0.411	0.272	0.272	0	0.411	0	0.521	0.411	0.272
3	0	0.445	0	0.232	0.232	0.445	0.35	0.445	0	0.35	0.232

Figure 2: Unigram TF-IDF

	excellent magazine	magazine bad	magazine great	magazine osfy	quality magazine
0	0	0	1	0	0
1	0	1	0	0	0
2	0	0	0.619	0	0.785
3	0.707	0	0	0.707	0

Figure 3: Bigram TF-IDF

Selecting an appropriate n-gram range can have a considerable impact on the analysis of NLP tasks. Unigrams, for example, may not successfully convey context (e.g., ‘not excellent’ versus just ‘excellent’), whereas bigrams, trigrams, or even higher-level n-grams may.

Unigram TF-IDF

The *sklearn* package makes summarising a corpus an easy task. The following code is an illustration of unigram analysis.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from tabulate import tabulate
import pandas as pd

# Sample documents
corpus = [
    "this magazine is great",          #Doc1
    "this magazine is bad",           #Doc2
    "the quality of this magazine is great", #Doc3
    "the name of this excellent magazine is OSFY" #Doc4
]

# Create TF-IDF vectorizer object
vectorizer = TfidfVectorizer()
# Fit and transform the corpus
vector = vectorizer.fit_transform(corpus)
# Convert to array and print
#print(vector.toarray())

# Vocabulary
feature_names = vectorizer.get_feature_names_out()

df = pd.DataFrame(vector.toarray(), columns=feature_names)
print(round(df,3))
```

Bigram TF-IDF

Here is an example with the same corpus using bigram.

```
# Create TF-IDF vectorizer object
vectorizer = TfidfVectorizer(
    max_features=500,
    stop_words='english',
    ngram_range=(2,2), # bigrams
    min_df=1,          # ignore rare words
    max_df=0.85        # ignore too frequent words
)

# Fit and transform the corpus
vector = vectorizer.fit_transform(corpus)
vector_df = pd.DataFrame(vector.toarray())

# Vocabulary
feature_names = vectorizer.get_feature_names_out()
#print(feature_names)
#print(tabulate(vector_df, headers=feature_names,
tablefmt="grid"))

# Convert to array and print
df = pd.DataFrame(vector.toarray(), columns=feature_names)
print(round(df,3))
```

Bigram gives us four meaningful contextual phrases, such as ‘quality magazine’, ‘excellent magazine’, ‘bad magazine’, and ‘osfy magazine’, together with the matching TF-IDF values for each of the four documents. These four bigrams also serve as indicative keywords for the corpus and point to OSFY magazine as a high-quality publication. **END** 🐧

By: Dr Dipankar Ray

The author has more than 30 years of experience in software development. He has published two books on MATLAB and medical image processing (published by PHI Learning and SPD).